

Revenue Monitor: Enterprise Feature Gate

Implementation Prompt for Ati Gravity

This document provides the complete specification for wiring the Revenue Monitor (Atrium Revenue Engine) so that it is fully functional and data-hydrated, but accessible **only to firms on the Custom/Enterprise tier**. Lower-tier firms (Starter/Core, Growth, Pro) will see the Revenue Monitor navigation item, but clicking it will surface an upgrade prompt rather than the full feature. This approach ensures the Revenue Monitor is "built out completely" and properly wired up, while preserving its value as a premium differentiator for enterprise clients. The document is structured to be handed directly to the Ati Gravity platform for implementation, with precise file references, code patterns, acceptance criteria, and risk considerations.

1. Current State of the Revenue Monitor

The Revenue Monitor (internally named **RevenueEngine**) is a five-tab property management dashboard that provides service charge tracking, financial ledger management, a unified inbox, automation centre, and a vacancy pipeline. It is the centrepiece of the Atrium product and the primary reason an enterprise property firm would upgrade from a lower tier. However, several critical gaps currently prevent it from functioning for real (non-demo) users, and there is no tier-level gate controlling access.

1.1 What Works

- The UI shell is fully built: **RevenueEngine.tsx** renders five tabs (Service Charges, Payments and Receipts, Unified Inbox, Automation Centre, Available Units) with KPI stats, badges, and responsive layout.
- The Convex backend is complete: **convex/sentry.ts** contains working mutations and queries for all five revenue tables (ledger_entries, service_charges, leads_pipeline, automation_logs, atrium_inbound_messages).
- The Convex schema defines all five tables with proper indexes (by_firm, by_unit, by_status, by_timestamp, etc.) in **convex/schema.ts**.
- Navigation exists: the Sidebar shows a RevenueEngineNavItem for property-mode firms, and BottomNav has a "Revenue" tab.
- A **FeatureGuard** already gates access to property-mode firms only (requiredProduct="property").

1.2 What Is Broken or Missing

- **No tier-level gate:** Any property firm, regardless of plan (Starter/Core through Enterprise), can access the full Revenue Monitor. There is no `canUseRevenueMonitor` flag in the `useFeatures` hook.
- **Revenue data is never hydrated for real users:** The `getFirmData` query in `convex/myFunctions.ts` fetches 29 tables but does NOT include `ledger_entries`, `service_charges`, `leads_pipeline`, or `automation_logs`. These arrays are always empty for non-demo users, even though the Convex tables exist and have data.
- **Only AtriumInbox works around this:** `AtriumInbox.tsx` bypasses `coreState` by calling `useQuery(api.sentry.getInboundMessages)` directly. The other four tabs (`ServiceChargeMonitor`, `LedgerManager`, `AutomationCenter`, `VacancyPipeline`) read from `coreState`, which is always empty.
- **No backend tier enforcement:** The `sentry.ts` mutations (`addLedgerEntry`, `upsertServiceCharge`, etc.) do not check the firm subscription plan before writing. A Starter-tier firm could theoretically write ledger entries.
- **SubscriptionPlan enum inconsistency:** `types.ts` defines `SubscriptionPlan` with Core, Growth, Pro, Ultimate, and Enterprise. But `tiers.ts` defines `TierId` as Core, Growth, Pro, and Enterprise (no Ultimate). This causes confusion in tier checks.

2. Architecture of the Enterprise Tier Gate

The goal is to make the Revenue Monitor fully functional and data-hydrated, but accessible only to Custom/Enterprise-tier firms. Lower-tier firms should see the Revenue Monitor in navigation (as a teaser), but clicking it should display an upgrade prompt. This creates a natural upsell path without hiding the feature entirely. The implementation follows existing patterns in the codebase: the `useFeatures` hook for frontend entitlement checks, the `FeatureGuard` component for product-level gates, and backend `ATRIUM_LIMITS` for server-side enforcement.

2.1 Tier Gate Layers

Layer	Mechanism	Location	Purpose
Frontend Entitlement	<code>canUseRevenueMonitor</code> boolean in <code>useFeatures</code> hook	<code>src/hooks/useFeatures.ts</code>	Controls UI rendering and upgrade prompts
Navigation Guard	Conditional rendering of nav items + upgrade CTA	<code>src/components/Sidebar.tsx</code> , <code>BottomNav.tsx</code>	Shows nav item but redirects non-Enterprise to upgrade
Route Guard	<code>TierGuard</code> wrapper around <code>RevenueEngine</code> route	<code>src/App.tsx</code> (atriumEngine case)	Blocks direct URL access for non-Enterprise firms
Backend Enforcement	Subscription plan check in <code>sentry.ts</code> mutations	<code>convex/sentry.ts</code>	Prevents API-level data manipulation by non-Enterprise firms
Data Hydration	Dedicated Convex queries per sub-component	Each Atrium sub-component	Feeds real data from Convex tables to Enterprise users

2.2 Entitlement Logic

The `canUseRevenueMonitor` flag should be defined in `src/hooks/useFeatures.ts` alongside the existing feature gates. The logic should mirror the existing `canUseAutomation` gate, which is already Enterprise-only. For Atrium/Property firms, the Revenue Monitor is the premium revenue management suite and should only unlock at the Enterprise/Custom tier. For Vega/Legal-only firms, it should remain inaccessible regardless of plan (the existing `FeatureGuard` handles product-level gating).

```
// In src/hooks/useFeatures.ts, add: canUseRevenueMonitor: isEnterprise, //
Enterprise/Custom only // Also add to checkFeatureAccess switch: case
'revenue_monitor': return isEnterprise;
```

IMPORTANT: The `isEnterprise` variable is already computed in `useFeatures` as: `isEnterprise = plan === SubscriptionPlan.Enterprise`. Since the user mentioned "Custom/Enterprise", and the `ATRIUM_TIERS` defines Enterprise as "Custom" pricing, this single check covers both labels. No additional tier level is needed.

3. Data Hydration Strategy

This is the most critical part of wiring the Revenue Monitor properly. Currently, the four main sub-components (`ServiceChargeMonitor`, `LedgerManager`, `AutomationCenter`, `VacancyPipeline`) all read from `coreState`, which is populated by `getFirmData`. However, `getFirmData` does NOT fetch from the revenue tables. The solution must hydrate these components with real Convex data without overloading the initial app load for non-Enterprise firms.

3.1 Recommended Approach: Dedicated Convex Queries Per Component

Rather than adding all five revenue tables to the already-heavy `getFirmData` query (which already fetches 29 tables and can take 2-5 seconds for large firms), the recommended approach is to add dedicated Convex queries inside each sub-component, following the pattern already established by `AtriumInbox`. This approach has several advantages: it avoids increasing the initial page load for all users; it only fetches data when an Enterprise user actually navigates to the Revenue Monitor; it is consistent with the Phase 2 data-loading strategy (`chatMessages`, `firmActivity`, and `researchMessages` were already moved to dedicated queries); and it allows each component to paginate and filter independently.

3.2 Required Query Additions

Component	Current Data Source	Required Convex Query	Convex Function
ServiceChargeMonitor	<code>coreState.serviceCharges</code> (empty)	<code>useQuery(api.sentry.getServiceChargesByFirm, {firmId})</code>	<code>getServiceChargesByFirm</code> (exists)
LedgerManager	<code>coreState.ledgerEntries</code> (empty)	<code>useQuery(api.sentry.getLedgerByFirm, {firmId})</code>	<code>getLedgerByFirm</code> (exists)
AutomationCenter	<code>coreState.automationLogs</code> (empty)	<code>useQuery(api.sentry.getAutomationLogs, {firmId})</code>	<code>getAutomationLogs</code> (exists)

VacancyPipeline	coreState.leadsPipeline (empty)	useQuery(api.sentry.getLeadsPipelineByFirm, {firmId})	getLeadsPipelineByFirm (NEEDS CREATION)
AtriumInbox	Direct query (works)	Already uses dedicated query	getInboundMessages (exists)

Important: The Convex backend already has most of the required query functions in **convex/sentry.ts**. The only missing query is `getLeadsPipelineByFirm`, which needs to be added. It should follow the same pattern as `getServiceChargesByFirm` and `getLedgerByFirm`, querying the `leads_pipeline` table with the `by_firm` index.

3.3 Frontend Refactor Per Component

Each of the four affected components needs a two-line refactor: replace the `coreState` data source with a dedicated Convex query. The existing Convex mutations (`addLedgerEntry`, `upsertServiceCharge`, etc.) will continue to work without changes because they write directly to the Convex tables, and the new queries will automatically pick up the updated data through Convex reactivity. Here is the pattern for each component:

```
// BEFORE (broken - reads empty coreState): const { coreState } = useCoreState();
const entries = coreState.ledgerEntries || []; // AFTER (working - uses dedicated
Convex query): const { coreState } = useCoreState(); const firmId =
coreState.firmDetails?.id || currentUser?.firmId || ""; const entries =
useQuery(api.sentry.getLedgerByFirm, { firmId }) ?? [];
```

IMPORTANT: After this refactor, the `coreState.ledgerEntries`, `coreState.serviceCharges`, `coreState.leadsPipeline`, and `coreState.automationLogs` properties become unused. They should be kept in the type definitions (to avoid breaking destructuring patterns) but can be documented as deprecated with a comment pointing to the dedicated queries.

4. Frontend Tier Gate Implementation

4.1 Step 1: Add `canUseRevenueMonitor` to `useFeatures`

Open `src/hooks/useFeatures.ts` and add the new entitlement flag. This follows the exact same pattern as the existing `canUseAutomation` and `canUseAuditLogs` gates, which are both Enterprise-only. The flag should appear in the returned object and in the `checkFeatureAccess` switch statement.

```
// Add to the Entitlements section of useFeatures(): canUseRevenueMonitor:
isEnterprise, // Add to checkFeatureAccess switch: case 'revenue_monitor': return
isEnterprise;
```

4.2 Step 2: Create a `TierGuard` Component

The existing `FeatureGuard` component handles product-level gating (property vs. legal), but there is no component for tier-level gating. Create a new **TierGuard** component that follows the same visual pattern as `FeatureGuard` but checks the subscription plan instead of the product mode. This component should show an upgrade prompt when the user does not have the required tier, with a call-to-action linking to the plan selection page. The component should accept a `requiredTier` prop (or a feature flag name) and use the `useFeatures` hook internally.

```
// src/components/TierGuard.tsx interface Props { feature: string; // e.g.
"revenue_monitor" children: React.ReactNode; } export const TierGuard: React.FC = ({
feature, children }) => { const { checkFeatureAccess, currentPlan } = useFeatures();
const { navigateTo } = useUI(); if (!checkFeatureAccess(feature as any)) { return (
navigateTo("settings")) } /> ); } return <>{children}; };
```

4.3 Step 3: Guard the RevenueEngine Route in App.tsx

In **src/App.tsx**, wrap the RevenueEngine route with both the existing FeatureGuard (product check) and the new TierGuard (tier check). The FeatureGuard must come first because it checks product mode, and the TierGuard wraps inside it to check the subscription tier.

```
// In App.tsx, replace the existing 'atriumEngine' case: case 'atriumEngine': return (
);
```

4.4 Step 4: Update Navigation Items

The Revenue Monitor should remain visible in navigation for all property-mode firms (as an upsell teaser), but clicking it when the firm is not on Enterprise should navigate to the atriumEngine view, which will then render the TierGuard upgrade prompt. This means no changes are needed to the navigation visibility logic itself, because the TierGuard in App.tsx will handle the blocking. However, the following refinements should be made for polish:

- **Sidebar.tsx:** Keep the RevenueEngineNavItem visible for all property firms. No code change needed since the TierGuard in App.tsx handles the gate.
- **BottomNav.tsx:** The permission function on the Revenue tab is currently `permission: () => true`. Consider adding a visual indicator (lock icon or "Enterprise" badge) for non-Enterprise firms, but do not hide the tab.
- **Optional enhancement:** Add a small "Enterprise" badge or lock icon next to the Revenue nav item when `canUseRevenueMonitor` is false. This provides a visual hint that the feature requires an upgrade before the user clicks it.

5. Backend Tier Enforcement

Frontend gates can be bypassed by direct API calls. The Convex mutations in `sentry.ts` must enforce the Enterprise-tier requirement server-side. This prevents a non-Enterprise firm from writing ledger entries, service charges, or pipeline leads through direct Convex client calls, even if they circumvent the frontend TierGuard.

5.1 Add Tier Check to Sentry Mutations

Every mutation in **convex/sentry.ts** that writes to the revenue tables should check the firm subscription plan before proceeding. The pattern should follow the existing `withFirmAuth` helper, but add a subscription plan check. Create a helper function that wraps the tier check logic.

```
// In convex/sentry.ts, add a helper: async function requireEnterpriseFirm(ctx: any,
firmId: string) { const firm = await ctx.db.get(firmId as any); if (!firm) throw new
Error("Firm not found"); if (firm.subscriptionPlan !== "Enterprise") { throw new
```

```
Error("Revenue Monitor requires an Enterprise subscription"); } return firm; } // Then
in each mutation handler, add as the first line: await requireEnterpriseFirm(ctx,
args.firmId);
```

5.2 Mutations That Need the Guard

Mutation	Table Written	Risk If Unguarded
addLedgerEntry	ledger_entries	Financial data manipulation
upsertServiceCharge	service_charges	Fraudulent charge creation
markChargeAsPaid	service_charges	Payment status manipulation
applyLatePenalty	service_charges	Penalty amount manipulation
addLeadToPipeline	leads_pipeline	Pipeline data pollution
advanceLeadStage	leads_pipeline	Stage manipulation
logAutomation	automation_logs	Audit log fabrication

Note: The read queries (getLedgerByFirm, getServiceChargesByFirm, etc.) do NOT need the Enterprise guard because the frontend TierGuard already prevents non-Enterprise users from reaching the components that call these queries. Adding server-side checks to read queries would be defensive but is not strictly necessary. The mutations are the priority because they modify data.

6. SubscriptionPlan Enum Fix

There is a naming inconsistency between **src/types.ts** and **src/constants/tiers.ts** that should be resolved before implementing the tier gate. The SubscriptionPlan enum in types.ts includes an "Ultimate" value that does not exist in the ATRIUM_TIERS or the TierId type. Meanwhile, the useFeatures hook uses isUltimate and isUltimateOrAbove variables that will never be true for Atrium/Property firms because no Atrium tier is named "Ultimate". This can cause subtle bugs where Enterprise Atrium firms are denied features that check isUltimateOrAbove instead of isEnterprise.

Source	Values	Issue
src/types.ts SubscriptionPlan	Core, Growth, Pro, Ultimate, Enterprise	Ultimate does not exist in any tier matrix
src/constants/tiers.ts TierId	Core, Growth, Pro, Enterprise	No Ultimate tier defined
useFeatures isUltimate	plan === SubscriptionPlan.Ultimate	Always false for Atrium firms

Recommended fix: Remove SubscriptionPlan.Ultimate from the enum (or alias it to Enterprise). Update all references to isUltimate and isUltimateOrAbove in useFeatures.ts to use isEnterprise instead. This ensures that the Enterprise tier check works correctly for both Atrium and Vega firms. The only code that currently uses isUltimate is the Sidebar for ALOA-X access, which should be updated to check isEnterprise.

7. KPI Stats Refactor in RevenueEngine.tsx

The top-level RevenueEngine.tsx component computes KPI stats (Collected, Outstanding, Defaults, Vacant) from coreState.ledgerEntries, coreState.serviceCharges, coreState.leadsPipeline, and coreState.automationLogs. After the data hydration refactor, these values will come from the dedicated Convex queries inside the sub-components, not from coreState. However, the KPI bar in RevenueEngine.tsx needs aggregate data to display.

7.1 Options

Option	Description	Pros	Cons
A: Aggregate queries	Add getCashFlowSummary and getRevenueKPIs queries in sentry.ts that return pre-computed KPIs	Efficient, single query for KPIs	New backend queries needed
B: Lift queries to parent	Move the four dedicated queries from sub-components up to RevenueEngine.tsx and pass data down as props	Parent has all data for KPIs	Breaks component independence, prop drilling
C: Context-based hydration	Create an AtriumRevenueContext that fetches all revenue data and provides it to children	Clean data flow, single fetch point	New context provider needed

Recommendation: Option A (Aggregate queries). This is the cleanest approach because it separates the KPI display from the detailed data. The getCashFlowSummary query already exists in sentry.ts. Add a similar getRevenueKPIs query that returns { collected, outstanding, defaults, vacant, criticalCount }. This keeps RevenueEngine.tsx lightweight and allows each sub-component to fetch its own detailed data independently with pagination and filtering.

```
// Add to convex/sentry.ts: export const getRevenueKPIs = query({ args: { firmId:
v.string() }, handler: async (ctx, { firmId }) => { const [ledger, charges, leads] =
await Promise.all([ ctx.db.query("ledger_entries").withIndex("by_firm", q =>
q.eq("firmId", firmId)).collect(),
ctx.db.query("service_charges").withIndex("by_firm", q => q.eq("firmId",
firmId)).collect(), ctx.db.query("leads_pipeline").withIndex("by_firm", q =>
q.eq("firmId", firmId)).collect(), ]); const now = new Date(); const collected =
ledger.filter(e => e.status === "cleared" && new Date(e.timestamp).getMonth() ===
now.getMonth()) .reduce((s, e) => s + e.amount, 0); const outstanding =
ledger.filter(e => e.status === "pending").reduce((s, e) => s + e.amount, 0); const
criticalCount = charges.filter(d => d.isDefaulter && (d.daysOverdue || 0) >
14).length; const vacant = leads.filter(l => l.stage !== "Closed").length; return {
collected, outstanding, defaults: criticalCount, vacant, criticalCount }; }, });
```

8. Implementation Order and Phases

The implementation should proceed in phases to minimize regression risk. Each phase is independently testable and can be deployed without waiting for subsequent phases. The order is designed so that the most impactful changes (data hydration) come first, followed by the tier gate, and finally polish and edge cases.

Phase 1: Data Hydration (Highest Impact)

This phase makes the Revenue Monitor actually work for users who have data. It has zero impact on existing functionality because it only adds new queries and changes internal data sources within the Revenue Monitor components. No other part of the app is affected.

Task	Files Changed	Risk	Acceptance Criteria
1. Add getLeadsPipelineByFirm query	convex/sentry.ts	Low	Query returns leads for a firm from leads_pipeline table
2. Add getRevenueKPIs query	convex/sentry.ts	Low	Query returns {collected, outstanding, defaults, vacant, criticalCount}
3. Refactor ServiceChargeMonitor to use dedicated query	src/components/atrium/ServiceChargeMonitor.tsx	Low	Service charges display real data from Convex
4. Refactor LedgerManager to use dedicated query	src/components/atrium/LedgerManager.tsx	Low	Ledger entries display real data from Convex
5. Refactor AutomationCenter to use dedicated query	src/components/atrium/AutomationCenter.tsx	Low	Automation logs display real data from Convex
6. Refactor VacancyPipeline to use dedicated query	src/components/atrium/VacancyPipeline.tsx	Low	Pipeline leads display real data from Convex
7. Refactor RevenueEngine KPI bar to use getRevenueKPIs	src/components/atrium/RevenueEngine.tsx	Medium	KPI stats show real aggregated numbers

Phase 2: Tier Gate (Enterprise Lock)

This phase restricts access to the Revenue Monitor to Enterprise-tier firms. It should be implemented after Phase 1 so that the gate is protecting a fully functional feature. The tier gate does not break any existing functionality for Enterprise users because it only adds an access check.

Task	Files Changed	Risk	Acceptance Criteria
1. Add canUseRevenueMonitor to useFeatures	src/hooks/useFeatures.ts	Low	Flag returns true only for Enterprise firms
2. Fix SubscriptionPlan.Ultimate inconsistency	src/types.ts, useFeatures.ts	Medium	isEnterprise correctly identifies Atrium Enterprise firms
3. Create TierGuard component	src/components/TierGuard.tsx (new)	Low	Shows upgrade prompt for non-Enterprise, renders children for Enterprise
4. Guard atriumEngine route with TierGuard	src/App.tsx	Low	Non-Enterprise firms see upgrade prompt instead of RevenueEngine
5. Add Enterprise badge to nav items	Sidebar.tsx, BottomNav.tsx	Low	Lock icon or "Enterprise" badge shown for non-Enterprise firms

Phase 3: Backend Enforcement

This phase adds server-side protection to prevent API-level circumvention. It is the final safety layer and should be deployed after the frontend gate is working. Each mutation check is independent and can be deployed incrementally.

Task	Files Changed	Risk	Acceptance Criteria
1. Add requireEnterpriseFirm helper	convex/sentry.ts	Low	Helper throws error for non-Enterprise firms
2. Guard addLedgerEntry	convex/sentry.ts	Low	Mutation rejects non-Enterprise firmId
3. Guard upsertServiceCharge	convex/sentry.ts	Low	Mutation rejects non-Enterprise firmId
4. Guard markChargeAsPaid	convex/sentry.ts	Low	Mutation rejects non-Enterprise firmId
5. Guard applyLatePenalty	convex/sentry.ts	Low	Mutation rejects non-Enterprise firmId
6. Guard addLeadToPipeline	convex/sentry.ts	Low	Mutation rejects non-Enterprise firmId
7. Guard advanceLeadStage	convex/sentry.ts	Low	Mutation rejects non-Enterprise firmId
8. Guard logAutomation	convex/sentry.ts	Low	Mutation rejects non-Enterprise firmId

9. Risk Analysis and Safeguards

The user specifically asked to "think through this carefully so that we do not spoil the good work done so far." The following risks have been identified and mitigated in the implementation plan.

Risk	Severity	Mitigation
Breaking existing demo mode	High	Demo mode uses ATRIUM_DEMO_APP_STATE which populates coreState. After Phase 1, the demo will still work because the sub-components will fall back to coreState when the Convex queries return no data (demo firm IDs do not match real Convex data). Verify demo mode after each phase.
Performance regression from new queries	Medium	Dedicated queries are only called when the user navigates to the Revenue Monitor, so they do not affect initial page load. Each query uses an indexed by_firm lookup. Add .take(100) limits and pagination for large firms.
SubscriptionPlan.Ultimate removal breaks existing code	Medium	Before removing Ultimate, search the entire codebase for all references. The only current usage is in Sidebar.tsx (ALOA-X access) and useFeatures.ts. Replace with isEnterprise. Keep Ultimate as a deprecated alias of Enterprise if needed for backward compatibility.

TierGuard breaks Unified/Komplet firms	Medium	Unified firms on Enterprise should access Revenue Monitor. The <code>isEnterprise</code> check covers this because Unified Enterprise maps to <code>SubscriptionPlan.Enterprise</code> . Test explicitly with a Unified Enterprise firm.
Backend guard blocks legitimate operations	High	The <code>requireEnterpriseFirm</code> check must use the same <code>subscriptionPlan</code> value that <code>useFeatures</code> checks. Test with a real Enterprise firm in staging. Add logging before throwing errors.
Race condition between tier upgrade and data access	Low	If a firm upgrades from Pro to Enterprise, the Convex <code>subscriptionPlan</code> field must be updated immediately. The frontend <code>useFeatures</code> re-reads from <code>coreState.firmDetails.subscriptionPlan</code> , which is fetched on login. A page refresh after upgrade will enable the feature.

10. Additional Improvements to Consider

While wiring up the Revenue Monitor behind the Enterprise gate, the following additional improvements should be considered. These are not strictly required for the tier gate, but they address issues found during the codebase audit and would improve the overall quality of the Revenue Monitor feature.

10.1 Receipt Generation at Unit Level

The current `LedgerManager` generates receipts at the property level, showing the property address rather than the tenant name. The user has explicitly requested that receipts should be at the unit level (for tenants), not the property level, and should show the tenant name. The `generateReceipt` function in `LedgerManager.tsx` should be updated to include the tenant name from the ledger entry (if `tenantId` is present) and to format the receipt for the specific unit/tenant rather than the whole property. Additionally, the "Issue Receipt" action should be added as a Quick Action in the property tracking view, not buried in the ledger.

10.2 Owner-Collects-Disburses Flow

The user has highlighted that the property manager does not collect rent directly. Instead, the property owner collects rent and then disburses the management fee to the property manager. The current system assumes the property manager collects rent, which is architecturally incorrect for the Nigerian property management market. The Revenue Monitor should support this flow by adding a "Disbursement" entry type to the ledger (alongside `rent`, `service_charge`, `penalty`, `deposit`) and adding a "Record Disbursement" action that records the management fee payment from the owner to the property manager. This should be considered a Phase 4 enhancement after the tier gate is working.

10.3 Unit-Level Edit Buttons

The user has requested unit-level edit buttons placed below the existing property edit button, beside each unit in the property tracking view. This is a UI improvement that would make the Revenue Monitor more usable for firms managing multi-unit properties. Each unit card should have an edit button that opens a unit-specific edit modal (not the current property-level modal that incorrectly shows a property selection dropdown).

10.4 Link Revenue Monitor to Financial and Analytics Pages

The Revenue Monitor currently operates in isolation. The user has requested that it should link to the Financial and Analytics pages so that clicking on a KPI (e.g., "Outstanding" or "Defaults") navigates to the corresponding filtered view in the analytics dashboard. This would require adding navigation callbacks to the KPI stat cards in RevenueEngine.tsx that call navigateTo with appropriate filter parameters.

10.5 Dual Payment System Synchronisation

The codebase audit identified that there are two independent payment tracking systems: the ledger_entries table and the property.rentPaymentHistory array. These are not synchronised, which means a payment recorded in one system does not appear in the other. The Revenue Monitor uses ledger_entries, while some property views use rentPaymentHistory. These should be consolidated so that ledger_entries is the single source of truth, and rentPaymentHistory is either deprecated or automatically synced from ledger entries. This is a larger refactor that should be planned separately.

11. Acceptance Test Plan

The following tests should be performed after each phase to verify that the Revenue Monitor Enterprise gate is working correctly and that no existing functionality has been broken.

11.1 Phase 1: Data Hydration Tests

- **Enterprise firm with data:** Log in as an Enterprise Atrium firm. Navigate to Revenue Monitor. Verify that all five tabs (Service Charges, Payments and Receipts, Unified Inbox, Automation Centre, Available Units) display real data from Convex tables.
- **Enterprise firm with no data:** Log in as an Enterprise Atrium firm with no ledger entries. Verify that the Revenue Monitor shows empty states (e.g., "No ledger entries yet") rather than loading spinners or errors.
- **Demo mode:** Log in with the demo firm ID. Verify that the Revenue Monitor still displays demo data correctly, falling back to coreState when Convex queries return empty.
- **Mutation round-trip:** In the LedgerManager, click "Record Entry" and add a new payment. Verify the entry appears immediately in the list (Convex reactivity). Navigate away and back. Verify the entry persists.
- **KPI accuracy:** Compare the KPI bar (Collected, Outstanding, Defaults, Vacant) with manual calculations from the raw Convex data. Verify they match.

11.2 Phase 2: Tier Gate Tests

- **Starter/Core firm:** Log in as a Starter-tier Atrium firm. Verify the Revenue nav item appears in Sidebar and BottomNav. Click it. Verify an upgrade prompt is shown instead of the Revenue Monitor. Verify the prompt mentions "Enterprise" and has a clear call-to-action.
- **Growth firm:** Same as above for Growth tier. Verify upgrade prompt is shown.

- **Pro firm:** Same as above for Pro tier. Verify upgrade prompt is shown.
- **Enterprise firm:** Log in as an Enterprise Atrium firm. Verify the Revenue Monitor loads fully with all tabs and data. No upgrade prompt should appear.
- **Unified Enterprise firm:** Log in as a Unified (Komplet) Enterprise firm. Verify the Revenue Monitor is accessible.
- **URL bypass:** As a Starter-tier firm, manually navigate to the atriumEngine route via URL. Verify the TierGuard still shows the upgrade prompt.
- **Upgrade flow:** As a Starter-tier firm, click the upgrade CTA. Verify it navigates to the settings/plan page. After upgrading to Enterprise (simulated), verify the Revenue Monitor becomes accessible after page refresh.

11.3 Phase 3: Backend Enforcement Tests

- **Direct API call from Starter firm:** Use the Convex client to call addLedgerEntry with a Starter-tier firmId. Verify the mutation throws an error with the message "Revenue Monitor requires an Enterprise subscription".
- **Direct API call from Enterprise firm:** Use the Convex client to call addLedgerEntry with an Enterprise firmId. Verify the mutation succeeds and the entry is written.
- **Read queries unguarded:** Verify that getLedgerByFirm, getServiceChargesByFirm, etc. still work for any firm tier (read queries should not be gated).

12. Summary

This document provides a complete, implementation-ready specification for wiring the Revenue Monitor behind an Enterprise/Custom tier gate. The key changes are: (1) hydrating the Revenue Monitor sub-components with real Convex data using dedicated queries instead of the empty coreState arrays; (2) adding a canUseRevenueMonitor entitlement flag to the useFeatures hook and a TierGuard component that shows an upgrade prompt for non-Enterprise firms; (3) enforcing the Enterprise requirement in the Convex sentry.ts mutations to prevent API-level circumvention; and (4) fixing the SubscriptionPlan.Ultimate inconsistency that could cause the Enterprise check to fail for Atrium firms.

The implementation is structured in three phases (Data Hydration, Tier Gate, Backend Enforcement) that can be deployed independently. Phase 1 has the highest impact because it makes the Revenue Monitor functional for the first time. Phase 2 adds the business logic that restricts it to Enterprise firms. Phase 3 adds the server-side safety net. Each phase includes specific acceptance criteria and risk mitigations to ensure that existing functionality is not broken.

The additional improvements in Section 10 (unit-level receipts, owner-collects-disburses flow, unit-level edit buttons, analytics linking, and dual payment system synchronisation) should be considered for subsequent iterations after the tier gate is stable. They represent the next level of maturity for the Revenue Monitor feature and should be prioritised based on user feedback.